Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Dersin yapısı
  - Bu ders de öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırma ile ilgili bir makaleyi özetledi; hoca sunumdan sonra makalenin yılını belirtti, sorular sordu ve yorum yaptı. Toplam yedi sunum yapıldı.
- Sunum 1: Doğal dil işleme için Bayesçi sıkıştırma (2018) (ekrandan, 00:10:00, 00:12:40, 00:13:20)
  - Konuşma tanıma ve soru cevaplama gibi görevlerde güçlü olan RNN'ler çok parametre içerir; sınırlı belleği olan cihazlarda kullanılmaları için sıkıştırılmaları gerekir.
  - RNN boyutunu azaltmanın üç yönelimi slaytta verildi: ağırlık matrislerinin tahmini, ağırlık matrislerinin hassasiyetinin azaltılması ve ağırlık matrislerinin seyreltilmesi. Çalışma üçüncüsüne odaklanır. Bu yönelimdeki en yaygın yaklaşım budamadır (pruning): ağırlıklar bir eşikte kesilir, ama eşiği belirleyen parametrelerin elle ayarlanması gerekir.
  - Bayesçi yaklaşımda ağırlıklar rastgele değişken olarak ele alınır. Sparse VD (seyrek varyasyonel dropout) bu ayarı eğitim sırasında kendisi yapar; daha az hiperparametre ister ve daha yüksek sıkıştırma oranı verir. Ağırlıkların önsel dağılımı log-uniform, sonsal dağılımı tam ayrışık (fully factorized) normal dağılım olarak alınır; sinyal-gürültü oranı düşük ağırlıklar sıfırlanıp atılır.
  - NLP görevlerinde gereksiz ağırlıkların çoğu gömme (embedding) katmanında toplanır. Kelime dağarcığını seyreltmek için çarpımsal ağırlıklar eklenir (Sparse VD-Voc); sıfırlanan ağırlıklar gereksiz kelimelerin atılması anlamına gelir.
  - Sonuçlar: IMDb'de Sparse VD-Voc %83,6 doğrulukla 12985 kat sıkıştırma ve 20000 yerine 292 kelimelik dağarcık; AGNews'te %89,2 doğrulukla 469 kat. Dil modellemede (Penn Treebank) sıkıştırma çok daha düşüktür: karakter düzeyinde 4,2x ve 3,53x, kelime düzeyinde 14,0x ve 11,1x.
- Sunum 2: Transformer ile kayıpsız veri sıkıştırma (2020) (ekrandan, 00:20:00, 00:27:20, 00:30:00, 00:36:30)
  - Kayıpsız sıkıştırma, orijinal verinin tam olarak geri kurulmasına izin veren algoritmalar sınıfıdır. İstatistiksel yöntemlerde (PAQ ailesi, PAQ8) bir tahminci sonraki sembolün olasılığını kestirir, ardından entropi kodlaması (aritmetik kodlama) yapılır.
  - Çalışmada PAQ8'in elle tasarlanmış özellikler kullanan tahmincisi yerine Transformer kullanılır; sıkıştırma, bir dil modelinin çevrimiçi öğrenilmesi problemine benzetilir. Veri seti enwik8'dir (İngilizce Vikipedi'nin 100 MB'lık kısmı); büyük harfler kaçış koduyla küçük harfe çevrilir (CMIX ön işlemesi).
  - Gözlemler: büyük modeller daha iyi oran verir; 12 katmandan derin modeller oranı iyileştirmez; daha büyük bağlam genelde daha iyidir, ama sıkıştırmanın başında kısa bağlamlı modeller daha iyidir; sonraki karakteri tahmin için en yararlı bilgi genelde birkaç önceki karakterdedir.
  - Başka yöntemler kod çözücüye sözlük göndermek zorunda olduğu için karşılaştırma, sözlüğün gzip -9 ile sıkıştırılmış boyutu da hesaba katılarak yapılır.
- Sunum 3: Metin iletimi için derin sinir ağını kayıplı sıkıştırma aşaması olarak kullanan kaynak kodlama (2020) (ekrandan, 00:42:30, 00:47:30, 00:48:30, 00:50:00)
  - İki aşamalı yöntemdir: önce FastText kelime gömmeleriyle kayıplı bir metin sıkıştırma, sonra kanonik Huffman kodlaması.
  - Her kelime için gömme uzayında kosinüs benzerliğine göre en yakın K kelime bulu-

nur; bunlardan en az karakterli olanı seçilir ve yanına kaçıncı komşu olduğu yazılır. Örneğin “automobile” yerine “car” ve sıra numarası gönderilir.
  - Kod çözmede Huffman tablosu (başlıkta gönderilir) ile ikili veri çözülür; kelime ve komşu sırası FastText ağına verilerek orijinal kelime bulunur.
  - Sonuçlar: başlıklı FastText+Huffman kısa cümlelerde kötüdür (5 kelimede normalize boyut 1’in üstündedir), cümle uzadıkça iyileşir (100 kelimede yaklaşık 0,45). Başlıksız FastText+Huffman yaklaşık 0,4 ile en iyisidir. K büyüdükçe sıkıştırma artar ama kelime hata oranı da artar; $K = 5$ için hata neredeyse sıfırdır.
- Sunum 4: Otokodlayıcı ile kayıplı görüntü sıkıştırma (2018) (ekrandan, 01:00:00, 01:01:35)
  - Görüntü pikselleri 255’e bölünerek 0-1 aralığına normalize edilir, simetrik bir evrişimli otokodlayıcının (CAE) kodlayıcısından geçirilir; ardından PCA döndürmesi, niceleme ve entropi kodlaması uygulanır.
  - Otokodlayıcı, girdiyi çıktıda yeniden üretmeyi öğrenir; ortadaki dar katman sıkıştırılmış temsildir. Eğitimden sonra ağ ortadan ikiye ayrılır: ilk yarı kodlayıcı, ikinci yarı kod çözücüdür.
  - Niceleme: $y' = \lceil 2^{B-1} \cdot y_{rot} \rfloor$, $B = 12$; kod çözücüde $\tilde{y} = y'/2^{B-1}$. Örnek: $0{,}123 \cdot 2^{11} = 251{,}904 \rightarrow 252$, $252/2^{11} = 0{,}123046875$. Niceleme hatası kadar düzgün (uniform) gürültü eğitimde eklenir.
  - Sonuç: Kodak veri setinde JPEG 2000’e göre %13,7 BD-rate azalması; 24 bpp’lik görüntüler yaklaşık 0,29 bpp’ye iner. Ortalama çalışma süresi CPU’da 2,29 s, GPU’da 0,67 s.
- Sunum 5: FOTA için program kodu sıkıştırma: CodeComp (2010) (ekrandan, 01:10:00, 01:13:10, 01:15:00, 01:20:00)
  - FOTA (Firmware Over The Air), cihaz yazılımının kablosuz güncellenmesidir. Güncelleme sırasında cihaz kullanılamadığı için kod çözme ve kodlamanın çok hızlı olması gerekir.
  - CodeComp, LZ ailesinin bir varyasyonudur. Bir talimat w, referans talimat r’den farklıysa w yerine $w \oplus r$ farkı kodlanır. En sık 256 fark bir fark tablosunda (DiffTable) tutulur, indeksi 1 byte’tır. Tabloda olmayan fark istisna değeri olarak saklanır. Hızlı çözme için tek tip uzunlukta kodlar kullanılır.
  - Yeniden kodlamayı hızlandırmak için delta dosyasındaki kopya bilgileri kullanılır; böylece delta boyutu artmaz.
  - Sonuçlar: kod çözme zlib’den yaklaşık 4 kat hızlı, sıkıştırma zlib’den yaklaşık %2 iyi. Kodlama hızı BPE’ye göre normalde yaklaşık 6 kat (379,22 / 60,70 KB/s), deltayla yaklaşık 1,8 kat (654,02 / 364,30 KB/s). BPE deltayı 740’tan 2140’a çıkarırken CodeComp 838’de bırakır.
- Sunum 6: Küçük mobil cihazlarda Shannon-Fano ve Huffman ile metin sıkıştırma (2017) (ekrandan, 01:30:00, 01:35:45, 01:40:00, 01:41:20)
  - Küçük mobil cihazlardan kasıt akıllı kartlardır; kaynakları çok kısıtlı olduğu için hafif algoritmalar gerekir. Akıllı kartlarda veri XML ya da PicoDBMS (Flat, Domain ve Ring Storage) ile saklanır.
  - Benzer çalışma olarak Burrows-Wheeler Transform (BWT) anlatıldı: üç aşamalıdır (BWT, Move-to-Front, entropi kodlayıcı); performansı girdinin karakteristiğine bağlıdır.
  - Calgary Corpus’tan 5 metin alınmış, birleştirilerek 1,1-3,8 MB’lık test dosyaları üretilmiştir. Sıkıştırma oranı Shannon-Fano’da %33,67-38,02, Huffman’da %34,19-39,08; Huffman biraz daha küçük dosya verir, Shannon-Fano biraz daha hızlıdır.
  - Sunan öğrenci makalenin sonuç bölümünde tabloyla çelişen rakamlar buldu: sonuçta ortalama süre Shannon-Fano 65,11 ms, Huffman 64,82 ms deniyor; tablonun ortalaması ise 79,57 ve 81,18 ms’dir (python ile doğrulandı). Makale yine de Shannon-Fano’yu önermiştir.
- Sunum 7: Sualtı nesnelerin interneti (IoUT) için DWT-CNN ile görüntü sıkıştırma (ekrandan, 01:50:00, 01:51:50, 01:54:30)
  - Sualtında görüntü iletimi zordur. Model iki CNN içerir: sualtı tarafında sıkıştırılmış temsil üreten C-CNN ve yüzey istasyonunda yeniden kurma yapan R-CNN; arada DWT tabanlı görüntü kodlayıcı (DWT/IDWT) vardır.
  - R-CNN 20 katmanlıdır (Conv+ReLU, Conv+BN+ReLU, Conv); ilk katmanda 64 adet $3 \times 3 \times c$ filtre, 2-19. katmanlarda 64 adet $3 \times 3 \times 64$ filtre vardır. Artık öğrenme (re-

sidual learning) ve toplu normalizasyon eğitimi hızlandırır; çözülen görüntü bikübik ara değerlemeyle büyütülür.
  - PSNR, S değeri arttıkça yükselir: Frame_002 için $S = 20$'de 32,29 dB, $S = 100$'de 51,55 dB.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınıflandırma mı, sıkıştırma mı?
  - Bayesçi sıkıştırma sunumunda hoca, atılan kelimelere geri dönülemeyeceğini, çalışmanın amacının sınıflandırma olduğunu ve sıkıştırmanın modelin küçültülmesi olarak kaldığını söyledi.
- Gömme vektörüyle nasıl sıkıştırma yapılır?
  - "car" 3 karakterdir, ama vektörü belki 100 boyutludur; hoca bunun veriyi büyütüp büyütmediğini sordu. Cevap: vektör gönderilmez, yalnız en kısa komşu kelime ve sıra numarası gönderilir. Hoca konum bilgisinin de yer kapladığını, ama 10 karakterlik "automobile" yerine yaklaşık 4 byte gittiğini belirtti.
- Makaleleri eleştirel okumak
  - Shannon-Fano sunumunda hoca, yayımlanmış makalelerde de hata olabileceğini, sonuçların tablolarla karşılaştırılması gerektiğini vurguladı.
- Shannon-Fano ne zaman Huffman'a yaklaşır?
  - Hoca, derste anlatıldığı gibi Shannon-Fano'nun olasılıklar 2'nin kuvvetleri olduğunda iyi çalıştığını hatırlattı. Önce BWT uygulamanın yardım edip etmeyeceğini düşündü, ama BWT'nin karakter frekanslarını değiştirmediğini belirtti.
- Görsel karşılaştırma
  - Otokodlayıcı sunumunda hoca örnek görüntüleri sütun sütun karşılaştırdı: çiçekli görüntüde JPEG 2000 (C sütunu) daha iyi görünüyor, papağanın beyaz bölgeleri tuhaf, çocuk görüntüsünde orijinalle fark görülmüyor.

Kısa Tekrar Notları

- Sparse VD: ağırlıklar rastgele değişken; log-uniform önsel, normal sonsal; düşük SNR'li ağırlıklar atılır; Voc sürümü kelime dağarcığını da seyreltir.
- Metin sınıflandırmada binlerce kat sıkıştırma (IMDb 12985x), dil modellemede 3,5-14 kat.
- Transformer ile kayıpsız sıkıştırma: tahminci + aritmetik kodlama; büyük model iyi, 12 katmandan derin fayda yok, bağlam büyüklüğü önemli.
- FastText + kanonik Huffman: kelime yerine en kısa komşu + sıra; K büyüdükçe sıkıştırma artar, hata artar.
- CAE: normalize (/255) → kodlayıcı → PCA → niceleme ($B = 12$) → entropi kodlayıcı; JPEG 2000'e göre %13,7 BD-rate kazanç.
- CodeComp: $w \oplus r$ farkı, 256 girişli DiffTable (1 byte indeks), istisna değeri, kopya işareti; zlib'den 4 kat hızlı çözme.
- Shannon-Fano vs Huffman: Huffman biraz daha küçük dosya, Shannon-Fano biraz daha hızlı; makalenin sonuç rakamları tabloyla çelişiyor.
- DWT-CNN: C-CNN + DWT kodlayıcı + R-CNN; PSNR S ile artar.

Detaylı Açıklamalar

- Bayesçi sıkıştırma sonuç tablosu (ekrandan, 00:13:20)
  - Metin sınıflandırma (doğruluk %, sıkıştırma, dağarcık, nöron): IMDb Original 84,1 / 1x / 20000 / 300-128; SparseVD 85,1 / 1135x / 4611 / 16-17; SparseVD-Voc 83,6 / 12985x / 292 / 1-8. AGNews Original 90,6 / 1x / 20000 / 300-512; SparseVD 88,8 / 322x / 5727 / 179-56; SparseVD-Voc 89,2 / 469x / 2444 / 127-32.
  - Dil modelleme (geçerleme, test, sıkıştırma, dağarcık, nöron): karakter düzeyinde (bit/karakter) Original 1,498 / 1,454 / 1x / 50 / 1000; SparseVD 1,472 / 1,429 / 4,2x / 50 / 431; SparseVD-Voc 1,4584 / 1,4165 / 3,53x / 48 / 510. Kelime düzeyinde (perplexity) Original 135,6 / 129,5 / 1x / 10000 / 256; SparseVD 115,0 / 109,0 / 14,0x / 9985 / 153; SparseVD-Voc 126,3 / 120,6 / 11,1x / 4353 / 207.

  - Sunan öğrenciye göre karakter düzeyinde her karakter önemli olduğu için seyreltme yapılamamış, kelime düzeyinde dağarcığın yarısından fazlası elenmiştir. Hocanın sorusu üzerine öğrenci, sınıflandırmada 300 birimlik gömme katmanı ve LSTM, dil modellemede 256 gizli birimli LSTM kullanıldığını söyledi.
- FastText tabanlı kayıplı kodlama örneği (ekrandan, 00:50:00)
  - "what is your phone number" cümlesi $K = 5$ ile "how3is1my2phone1number1" olarak sıkıştırılır ve doğru açılır. $K = 30$ ve $K = 50$ ile "it40is1my2fax12" olur ve "what is your phone" olarak açılır; "number" kaybolmuştur.
  - "paper money has been used for over a thousand years" cümlesi $K = 5$, 30 ve 50 ile farklı biçimlerde sıkıştırılır (örneğin $K = 5$: "paper1cash2is5has2used1of4over1a1dozen8days4" ama üçünde de doğru açılır.
  - K grafiğinde (ekrandan, 00:48:30) lejant: $K = 5$ Compress 82, $K = 10$ Compress 77, $K = 20$ Compress 74, $K = 30$ Compress 73, $K = 50$ Compress 70. 40 kelimelik cümlede kelime hata oranı $K = 50$ için 3'ün üstüne çıkar.
- CodeComp kodlama örneği (ekrandan, 01:13:10)
  - Referans sırası: `00 0F 01 02 | 21 5F | 11 F1 F2 0A 00 00 FF FE 00 01 FF F0`. Kodlama sırası: `00 0E F0 02 | 11 F1 F2 0A 00 00 FF FE 00 01 FF F0`.
  - İkinci byte'ta 0E ile 0F farklıdır; fark ($0F \oplus 0E$) DiffTable'da bulunduğu için istisna işareti 0 olur ve tablonun indeksi p saklanır. Üçüncü byte'ta F0 ile 01 farklıdır ve fark tabloda yoktur; istisna işareti 1 olur ve değer (F0) olduğu gibi saklanır.
  - Kopya işareti dizisi 1, 0, 0, 1, 1, ..., 1'dir: eşit byte'larda 1, farklı byte'larda 0. Dist işareti: 0, 1, 0, 1, 1, 1, 1, 1.
- Geriye doğru kodlama örneği (ekrandan, 01:15:00)
  - Adres 20-30'daki 0F FF FA E3 BF BD kodlanacaktır. Şema 1 ikişerli bloklarla çalışır: (20-22) (16-18)'deki 0F FF'yi, (24-26) (12-14)'teki FA E2'yi, (28-30) (8-10)'daki BF BD'yi referans alır. Şema 2'de (20-30) bloğu tek parça olarak (0-10)'daki 0F FF FA F3 BF BD'yi referans alır. İki şemada da E3 farklı bir değerle (E2 ya da F3) karşılaştırılır ve fark üzerinden kodlanır.
- Shannon-Fano ve Huffman tabloları (ekrandan, 01:35:45, 01:40:00)
  - Dosya boyutu (byte, SF/H): test1 1941646 / 1908295; test2 745249 / 724476; test3 971274 / 960170; test4 1165046 / 1145025; test5 2568026 / 2548143.
  - Kodlama süresi (ms, SF/H): 101,09 / 103,45; 54,86 / 47,81; 52,52 / 63,20; 62,00 / 71,43; 127,38 / 120,02. Ortalamalar 79,57 ve 81,18 ms; Shannon-Fano ortalamada yaklaşık %2 daha hızlıdır (python ile doğrulandı).
  - Makalenin bulgular bölümü Huffman'ın boyut açısından yaklaşık %1,66, Shannon-Fano'nun süre açısından yaklaşık %2,3 daha iyi olduğunu söyler. Sonuç bölümü ise farkları %1,22 ve %1,12 olarak verir ve ortalama süreyi Shannon-Fano için 65,11 ms, Huffman için 64,82 ms yazar; bu rakamlar tabloyla uyuşmaz.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.